Week 4 · Lab 1

Review — Operators and Conversions: Mixing Types, Conditions and `if/else`

Implicit conversion · `static_cast` · Relational and logical operators · `if` · `if-else` · `else if` · nested `if`

Week: 4 — first session (review)
Duration: 2 hours 15 minutes
Format: review of the Week 3 lecture, then practice
Tools: computer with CLion (or any C++17 compiler); pen for Sections A–H
Prerequisite: Week 3 lecture; Labs 3–4

Name:

Group:

Date:

Learning outcomes

By the end of this lab you should be able to:

- **LO1.** Predict the type and value of an expression that mixes `int`, `double`, `char` and `bool`.
- **LO2.** Explain what is lost when a value is converted to a narrower type, and use `static_cast` to convert on purpose.
- **LO3.** Evaluate relational (`< <= > >= == !=`) and logical (`&& || !`) expressions, including short-circuit evaluation.
- **LO4.** Translate an English condition into a correct C++ boolean expression.
- **LO5.** Write `if`, `if-else` and `else if` chains with correct braces and condition order.
- **LO6.** Read nested `if` statements correctly, including an `else` that belongs to the nearest `if`.
- **LO7.** Recognise the classic selection bugs: `=` for `==`, a stray `;`, missing braces, mis-ordered conditions.

This is the review session of Week 4: it goes through the Week 3 lecture again (*operators and conversions; selection statements; if/else*) and solves problems on it together. It is also the revision lab for Quiz 1 (Week 4). The ternary operator and `switch` are not used here; they come with the Week 4 lecture.

How this lab is organised

This is a **review** session: everything here was in the Week 3 lecture, and Labs 3–4 used most of it. **Part I** goes through each idea again with a worked example; after most ideas there is a short **Quick check** that you solve immediately, before moving on. In **Part II** the instructor solves the *first item* of Sections A–F on the board together with the class; you finish the rest yourself. Sections A–H are done *on paper, without a computer*; Sections I–L are done in **CLion**. Quiz 1 is this week — Sections A, C, E and the `if/else` programs are the best revision for it.

Block	Content		Time
–	Introduction and learning outcomes		5 min
PART I — THEORY AND WORKED EXAMPLES			40 min
§1	Implicit conversion inside expressions		5 min
§2	Conversion on assignment: what gets lost		4 min
§3	Explicit conversion: `static_cast`		6 min
§4	Relational operators and `bool`		4 min
§5	Logical operators and short-circuit evaluation		7 min
§6	`if`, `if-else`, `else if`		7 min
§7	Nested `if` and the dangling `else`		5 min
§8	Common mistakes		2 min
–	*Short break*		5 min
PART II — PRACTICE			80 min
A–B	Mixed-type expressions; casts	*paper*	12 min
C–D	Evaluate conditions; write conditions	*paper*	12 min
E	Predict the output: `if/else`	*paper*	8 min
F	Find the mistake	*paper*	6 min
G–H	True/false and multiple choice	*paper*	8 min
I	Correct average	*CLion*	8 min
J	Even, odd and divisible	*CLion*	7 min
K	Grade calculator	*CLion*	10 min
L	BMI classifier	*CLion*	9 min
M	Bonus challenges *(not timed)*		—
–	Final checklist and wrap-up		5 min
Total			**2 h 15 min**

> **How your work is checked.** Nothing is uploaded — everything is checked here, during the lab. Write the answers to Sections A–H on this sheet. Do Sections I–L in CLion, one `.cpp` file per task, and test every program with *all* the inputs listed in the task — for programs with `if`, a single test proves almost nothing. **When you have finished, raise your hand** and show the TA your answer sheet and each program running. The TA marks the score table on the last page.

PART I — Theory and worked examples

1 Implicit conversion inside expressions

In Lab 3, `7 / 2` gave `3` but `7 / 2.0` gave `3.5`, and `1.0 * a / b` fixed an average. This section explains the rule behind all three.

The usual arithmetic conversions

A binary operator such as `+` or `/` works on two values *of the same type*. When the types differ, C++ converts the "smaller" one **automatically** before computing:

`bool, char` → `int` → `long long` → `float` → `double`

`bool` and `char` always become `int` first (`true` is 1, `'A'` is 65). Then, if one operand is `double`, the other becomes `double`. The **result has the common type**.

Conversion happens *per operator*, one step at a time, left to right — not once for the whole line:

Worked example 1.1 — the same numbers, two orders

Expression	Step by step	Result
`5 / 2 * 2.0`	`5 / 2` is `int` → 2; then `2 * 2.0` → `4.0`	4 (double)
`2.0 * 5 / 2`	`2.0 * 5` → `10.0`; then `10.0 / 2` → `5.0`	5 (double)
`'A' + 2`	`'A'` → 65; `65 + 2`	67 (int)
`true + true`	`1 + 1`	2 (int)

The first line has a `double` in it and still loses the `.5`: by the time `2.0` is involved, the integer division has already happened. **Where** the `double` appears matters.

Quick check 1. Give the type and value of `9 / 4 + 0.5` and `9 / 4.0 + 0.5`.

2 Conversion on assignment: what gets lost

When you store a value in a variable of a different type, it is converted to the *variable's* type. Going to a wider type is safe; going to a narrower one can lose information **silently**.

Statement	Stored	What happened
`double d = 7;`	`7.0`	`int` → `double`: exact.
`int i = 3.99;`	`3`	`double` → `int`: fraction **truncated**, not rounded.
`int n = -3.7;`	`-3`	Truncation is **towards zero**.
`double e = 7 / 2;`	`3.0`	The right side is computed first, as `int`. The variable's type is too late to help.
`char c = 66;`	`'B'`	`int` → `char`: the code 66 is `'B'`.
`bool b = -5;`	`true`	Any non-zero number becomes `true`; only `0` becomes `false`.

Brace initialisation refuses to lose data: `int i{3.99};` is a *compile error* ("narrowing conversion"), while `int i = 3.99;` compiles with at most a warning. Turn warnings on and read them.

Quick check 2. `int total = 17; double avg = total / 2;` What is `avg`, and why is changing `avg` to `float` no fix?

3 Explicit conversion: `static_cast`

When you *want* a conversion, say so:

`static_cast<type>(expression)`

```
1 int sum = 17, count = 2;
2 double a = static_cast<double>(sum) / count;   // 8.5  -- cast ONE operand, then divide
3 double b = static_cast<double>(sum / count);   // 8.0  -- divides first, casts the 8
4 int    r = static_cast<int>(2.7 + 0.5);        // 3    -- round a positive number
5 char   n = static_cast<char>('a' + 1);         // 'b'  -- back from int to char
```

Cast the operand, not the result. In line 2 the parentheses make the division happen *first*, in integers, and only the finished `8` is converted. The fix is always the same: cast *one of the operands* before the operator runs.

The old C-style form `(double)sum` does the same job and you will see it in older code. Prefer `static_cast`: it is easy to search for and it refuses some dangerous conversions.

Worked example 3.1 — rounding to the nearest integer

Converting to `int` truncates. To *round* a positive value, add 0.5 first: `static_cast<int>(84.33 + 0.5)` = 84, `static_cast<int>(70.67 + 0.5)` = 71. (For negative numbers you would subtract 0.5 — or use `round` from `<cmath>`, which you will meet later.)

Quick check 3. `int a = 13, b = 4;` Which of these print `3.25`? (i) `a / b` (ii) `static_cast<double>(a) / b` (iii) `static_cast<double>(a / b)` (iv) `a / static_cast<double>(b)`

4 Relational operators and `bool`

A **relational** operator compares two values and produces a `bool`: `true` or `false`. Printed with `cout`, they appear as `1` and `0`.

Op	Meaning	Op	Meaning
<	less than	>	greater than
<=	less than or equal	>=	greater than or equal
==	**equal to**	!=	not equal to

= assigns, == compares. Lab 2 promised this would be the most common bug, and it is: `if (x = 0)` stores 0 in `x` and then tests `0`, which is `false` — so the branch never runs and `x` has been destroyed. It compiles. Read every `=` inside a condition twice.

Do not compare real numbers with ==. `0.1 + 0.2 == 0.3` is `false` in C++: neither side is stored exactly in binary (Lab 1). Also, `9 / 2 == 4.5` is `false`, because `9 / 2` is `4`.

Quick check 4. With `int a = 4, b = 9;` what do `cout << (a != b);` and `cout << (b / 2 == a);` print?

5 Logical operators and short-circuit evaluation

p	q	p && q	p || q	!p
F	F	F	F	T
F	T	F	T	T
T	F	F	T	F
T	T	T	T	F

`&&` (AND) is true only when **both** sides are true.
`||` (OR) is true when **at least one** side is true.
`!` (NOT) flips a single value.

These are the `AND`, `OR`, `NOT` of Lab 2's pseudocode.

Precedence, highest first: `!` > `* / %` > `+ -` > `< <= > >=` > `== !=` > `&&` > `||` > `=`. So `a + 1 > b && c == 0` means `((a + 1) > b) && (c == 0)`. When in doubt, add parentheses.

Short-circuit evaluation

C++ evaluates `&&` and `||` **left to right and stops as soon as the answer is known**. If the left side of `&&` is false, the right side is never evaluated; if the left side of `||` is true, the right side is never evaluated. This lets the left side *guard* the right side:

`if (count != 0 && total / count > 50)` — never divides by zero.

Swap the two halves and the program crashes when `count` is 0.

Worked example 5.1 — "between" needs two comparisons

Mathematics writes $0 \le x \le 100$. C++ does **not** read `0 <= x <= 100` that way: it evaluates `(0 <= x)`, gets `0` or `1`, and compares *that* with 100 — which is always true. For `x = 150` the expression is true. (Recent compilers warn about this; some refuse to compile it.)

Correct: `x >= 0 && x <= 100` Outside the range: `x < 0 || x > 100`

The second form is the *negation* of the first: NOT (A AND B) is (NOT A) OR (NOT B).

Quick check 5. Write a condition that is true when `ch` is a lowercase letter.

6 Selection: `if`, `if-else`, `else if`

The pseudocode of Lab 2 becomes C++ almost word for word. The condition goes in **parentheses**; the block goes in **braces**.

```cpp
if (score >= 60) {
    cout << "Pass" << endl;
} else {
    cout << "Fail" << endl;
}
```

```cpp
if (n > 0) {
    cout << "Positive";
} else if (n < 0) {
    cout << "Negative";
} else {
    cout << "Zero";
}
```

An `else if` chain tests conditions **in order** and runs only the *first* block whose condition is true — exactly as in Lab 2, Worked example 4.1. Put the strictest test first.

Braces are not optional in practice. Without braces, `if` controls exactly *one* statement, whatever the indentation says:

```cpp
if (x > 10)
    cout << "big ";
    cout << "really big";   // NOT inside the if: always runs
```

And a semicolon right after the condition, `if (x > 10);`, is an *empty* statement: the `if` controls nothing and the next line always runs. Rule for CS102: **always use braces**, even for one line.

Worked example 6.1 — a validated grade band

```cpp
double score;
cin >> score;
if (score < 0 || score > 100) {
    cout << "Invalid score" << endl;
} else if (score >= 85) {
    cout << "Distinction" << endl;
} else if (score >= 60) {
    cout << "Pass" << endl;
} else {
    cout << "Fail" << endl;
}
```

Test with values chosen to hit every branch *and* every boundary:

`-1, 101`	Invalid score
`100, 85`	Distinction
`84.9, 60`	Pass
`59.9, 0`	Fail

Invalid input is rejected *first*, so later tests may assume $0 \le score \le 100$.

Quick check 6. `int x = 5;` What does this print? `if (x > 3) cout << "A"; else if (x > 1) cout << "B"; else cout << "C";`

7 Nested `if` and the dangling `else`

A block of an `if` may contain another `if`. Nest when the second question only makes sense *after* the first one has been answered:

```cpp
if (age >= 18) {
    if (hasLicence) {
        cout << "You may drive" << endl;
    } else {
        cout << "Get a licence first" << endl;
    }
} else {
    cout << "Too young to drive" << endl;
}
```

Three different answers, so three blocks. The inner `if` is asked only for adults.

When there is **no** `else`, `if (a) { if (b) { X } }` is the same as `if (a && b) { X }` — the shorter form is usually clearer.

The dangling-else rule

Without braces, an `else` belongs to the **nearest** `if` above it that has no `else` yet — whatever the indentation says. The compiler ignores indentation completely.

Worked example 7.1 — indentation that lies

```cpp
int n = -4;
if (n > 0)
    if (n % 2 == 0)
        cout << "positive even";
else
    cout << "not positive";
cout << "." << endl;
```

The writer meant the `else` for `n > 0`. It actually pairs with `n % 2 == 0`. For `n = -4` the outer test is false, the whole inner `if-else` is skipped, and the program prints only `.`

For `n = 7` it prints `not positive.` — wrong again.

Fix: braces around the inner `if`.

Quick check 7. What does this print for `t = 30`? And for `t = 10`? `if (t > 20) if (t > 35) cout << "hot"; else cout << "mild"; cout << "!";`

8 Common mistakes — and how to catch them

Mistake	Symptom and fix
`static_cast<double>(a / b)`	Cast applied after integer division. Cast an operand: `static_cast<double>(a) / b`.
`if (x = 5)`	Assignment, always true (for non-zero). Use ==.
`if (cond);`	Empty statement; the block below always runs. Delete the `;`.
`0 <= x <= 100`	Always true. Write `x >= 0 && x <= 100`.
`if (n % 2 == 1)` for "odd"	Fails for negative odd numbers, where `n % 2` is `-1`. Test `n % 2 != 0`.
Loosest test first in an `else if` chain	Later branches are unreachable. Strictest first.
Nested `if` without braces	The `else` pairs with the nearest `if`, not the one the indentation suggests. Use braces.

PART II — Practice

A Mixed-type expressions

[6 min] (8 points)

Give the **type** of the result and its **value**. ($\frac{1}{2}$ + $\frac{1}{2}$ each)

	Expression	Type	Value
1	`7 / 2`		
2	`7 / 2.0`		
3	`7 % 3 * 1.5`		
4	`'A' + 2`		
5	`'a' - 'A'`		
6	`true + true`		
7	`5 / 2 * 3.0`		
8	`3.0 * 5 / 2`		

B Conversions and casts

[6 min] (8 points)

Write what the program prints. (4 per line; −1 for each wrong value)

```
int    i = 3.99;
double d = 7 / 2;
double e = static_cast<double>(7) / 2;
double f = static_cast<double>(7 / 2);
int    r = static_cast<int>(2.7 + 0.5);
int    n = -3.7;
char   c = 66;
int    k = 'z' - 'a' + 1;
bool   b = -5;
cout << i << " " << d << " " << e << " " << f << endl;          // line 1
cout << r << " " << n << " " << c << " " << k << " " << b << endl;   // line 2
```

Line 1: _____ Line 2: _____

C Evaluate the condition

[6 min] (8 points)

With `int a = 4, b = 9, c = 0;` and `char ch = 'm';` write `1` (true) or `0` (false). (1 each)

1. `a < b` _____

2. `a == 4 && b < 5` _____

3. `a > 10 || b > 5` _____

4. `!c` _____

5. `!(a < b) || c` _____

6. `c != 0 && b / c > 1` _____

7. `a % 2 == 0 || b % 2 == 0` _____

8. `!(a > b) && ch >= 'a' && ch <= 'z'` _____

D Write the condition

[6 min] (8 points)

Write one C++ boolean expression for each description. (1 each)

1. `x` is between 1 and 100 inclusive. _____

2. `ch` is an uppercase letter. _____

3. `n` is even but not divisible by 10. _____

4. `age` is outside the range 18–60. _____

5. `a`, `b` and `c` are all different.

6. `a`, `b`, `c` can be the sides of a triangle.

7. `score` is *not* in $[0, 100]$, written using `!`.

8. `h` and `m` form a valid 24-hour time.

E Predict the output: `if/else`

[8 min] (10 points)

Each fragment is separate. Write exactly what it prints. (2 each)

a)

```
int x = 5;
if (x > 10);
    cout << "big" << endl;
cout << "done" << endl;
```

b)

```
int x = 5;
if (x > 10)
    cout << "big ";
    cout << "really big" << endl;
```

c)

```
int x = 5;
if (x = 0)
    cout << "zero" << endl;
else
    cout << "nonzero, x = " << x << endl;
```

d)

```
int t = 28;
if (t > 10)
    cout << "warm" << endl;
else if (t > 25)
    cout << "hot" << endl;
else
    cout << "cold" << endl;
```

e)

```
int a = 3, b = 7;
if (a > 5)
    if (b > 5)
        cout << "A" << endl;
else
    cout << "B" << endl;
cout << "C" << endl;
```

F Find the mistake

[6 min] (8 points)

Each fragment has exactly one fault. Name it, say whether it is a syntax or a logic error, and write the fix.

a) (2) Intended: print whether `n` is odd or even, for any integer.

```
if (n % 2 == 1) { cout << "odd"; } else { cout << "even"; }
```

b) (2) Intended: the exact average of three integer scores.

```cpp
double avg = static_cast<double>(sum / 3);
```

c) (2)

```cpp
if (score >= 60)
    cout << "Pass";
    cout << " - well done";
else
    cout << "Fail";
```

d) (2) Intended: print `"Big ratio"` when `b` is non-zero and `a / b > 2`.

```cpp
if (b != 0 || a / b > 2) { cout << "Big ratio"; }
```

G True or false

[4 min] (6 points)

Mark each statement **T** or **F**. For every **F**, write one short line saying why.

1. In `3 + 2.5`, the `3` is converted to `double` before the addition. ____

2. `int x = 9.99;` stores `10`. ____

3. `static_cast<double>(7 / 2)` equals `3.5`. ____

4. In `p && q`, if `p` is false then `q` is not evaluated. ____

5. `if (x = 5)` tests whether `x` equals 5. ____

6. Without braces, an `if` controls only the next single statement. ____

H Multiple choice

[4 min] (6 points)

Circle one answer per question.

1. `cout << 10 / 4.0;` prints
(a) 2 (b) 2.5 (c) 2.0 (d) 3

2. `!(3 > 2 && 5 < 4)` is
(a) true (b) false (c) error (d) 5

3. Which tests "x is 0 or negative"?
(a) `x =< 0` (b) `x <= 0`
(c) `x < 0 && x == 0` (d) `x => 0`

4. `static_cast<int>(-2.9)` is
(a) −3 (b) −2 (c) 2 (d) −2.9

5. Which operator has the **lowest** precedence?
(a) && (b) == (c) || (d) +

6. `int a = 3, b = 4; double r = a / b;` `r` is
(a) 0.75 (b) 0 (c) 1 (d) error

I Correct average

[8 min] (8 points)

Task I — one `.cpp` file

Read three integer scores (0–100). Print their sum; the exact average (use `static_cast`); the average rounded to the nearest whole number (use `static_cast<int>` and +0.5); the percentage of the maximum 300; and, for comparison, the "wrong average" `sum / 3`.
Test input: `85 90 78`

```
Sum:            253
Average:        84.3333
Rounded:        84
Percentage:     84.3333%
Wrong average: 84
```

With `70 71 71`: Average _____ Rounded _____ Wrong average _____

J Even, odd and divisible

[7 min] (8 points)

Task J — one `.cpp` file

Read two integers `a` and `b`. Print whether `a` is even or odd (it must work for negative numbers). Then, if `b` is 0, print `Cannot divide by zero`; otherwise print whether `a` is divisible by `b` and the exact quotient `a / b`.

Input `17 5`

```
17 is odd
17 is not divisible by 5
17 / 5 = 3.4
```

Input `-12 4`

```
-12 is even
-12 is divisible by 4
-12 / 4 = -3
```

Input `7 0`

```
7 is odd
Cannot divide by zero
```

Also test `-7 2`. First line: _____

K Grade calculator

[10 min] (12 points)

Task K — one `.cpp` file

Read a score (`double`). If it is outside $[0, 100]$ print `Invalid score`. Otherwise print the letter grade and the result:

Score	≥ 90	80–89.99	70–79.99	60–69.99	< 60
Grade	A	B	C	D	F

followed by `Result: Pass` for A–D or `Result: Fail` for F. Store the letter in a `char` variable.
Test all of these and write the grade (or *Invalid*):

Input	95	89.5	60	59.99	100	-1	101
Output							

L BMI classifier

[9 min] (10 points)

Task L — one `.cpp` file

Read a weight in kilograms (`double`) and a height in **centimetres** (`int`). Convert the height to metres, compute BMI = weight/height^2, print it and classify it: below 18.5 *Underweight*; below 25 *Normal*; below 30 *Overweight*; otherwise *Obese*.

Input	Expected output	Your output
`70 175`	`BMI: 22.8571 -> Normal`	
`50 180`	`BMI: 15.4321 -> Underweight`	
`80 175`		
`90 170`		

A classmate writes `double heightM = heightCm / 100;`. What does their program print for `70 175`, and why?

M Bonus challenges (up to 10 points, not timed)

Attempt these only after everything above is finished. Write and test each in CLion.

1. (3) **Cinema ticket (nested `if`).** Read an age (`int`) and a `char` that is `y` if the customer is a student and `n` otherwise. Children under 7 enter free; customers aged 65 or more pay 20000 so'm; everyone else pays 45000 so'm, or 30000 so'm with a student card. Print the price. Test `5 n`, `70 y`, `19 y`, `19 n`.

2. (4) **Character type and case flip.** Read one `char`. Print `uppercase`, `lowercase`, `digit` or `other`; for a letter also print it in the other case, for a digit print its numeric value. Use `'a' - 'A'` and `static_cast<char>` — no library functions. Examples: `g` → `lowercase, other case: G`; `7` → `digit, value 7`.

3. (3) **One second later.** Read a valid time as three integers `h m s` (24-hour clock) and print the time one second later in the same form. Test `10 15 30`, `10 15 59`, `10 59 59`, `23 59 59`.

Final checklist [5 min]

Before you raise your hand, confirm each line. Tick the box.

- ☐ Your **name** and **group** are on the front page.
- ☐ Sections A–H are answered **on paper**, before you used a computer to check anything.
- ☐ Every division that must be real has a `double` **operand** — not just a `double` result variable.
- ☐ Every `if` and `else` block has **braces** (so no `else` can dangle), and no condition is followed by `;`.
- ☐ Every comparison for equality uses `==`, not `=`.
- ☐ Every `else if` chain tests the **strictest** condition first.
- ☐ Each program was run with **every** test input in its task, including the boundaries.
- ☐ You **raised your hand** and showed the TA this sheet and each program running.

Section	Available	Earned
A Mixed-type expressions	8	
B Conversions and casts	8	
C Evaluate the condition	8	
D Write the condition	8	
E Predict the output: if/else	10	
F Find the mistake	8	
G True or false	6	
H Multiple choice	6	
I Correct average	8	
J Even, odd and divisible	8	
K Grade calculator	12	
L BMI classifier	10	
Total	**100**	
M Bonus challenges	+10	

Checked by TA: _____

What comes next

- **Quiz 1** is this week. It covers Weeks 2–3: number systems, types, `/` and `%`, casts, conditions with short-circuit, and `if`/`else` including the dangling `else`.
- **Week 4 · Lab 2** is the graded practice session: problems to solve in CLion, checked at your desk.
- The **Week 4 lecture** adds the **ternary** operator `?:` and `switch`/`case`.

The habit worth keeping from today: **test every branch and every boundary**. A program with four branches that you ran once has been tested 25 % at best.

End of Week 4 · Lab 1